

Karten App

Aufbau, Architektur und Code-Dokumentation der Ionic/Vue/Capacitor-Anwendung „Karten App“ – Adresssuche, Standortanzeige und Suchverlauf als Android-App.

Repository	Fallstudie_Genz / karten-a-pp
Kontext	DHBW-Fallstudie „Genz“
Plattform	Android (APK), gebaut via Capacitor
Erstellt für	philong95@hotmail.de
Stand	11.08.2026

Vue 3

TypeScript

Ionic Framework

Vite

Capacitor 8

Leaflet + OpenStreetMap

Nominatim API

Android / Gradle

— Inhalt

1 Überblick & Tech-Stack

2 Architektur – wie die Schichten zusammenhängen

3 Projektstruktur (Ordner & Dateien im Überblick)

4 State Management & Datenfluss (mapStore)

5 Datei-Dokumentation – Frontend (src/)

6 Datei-Dokumentation – Android-Projekt

7 Build- & Release-Workflow

8 Glossar wichtiger Begriffe

1 Überblick & Tech-Stack

Die „Karten App“ ist eine mobile Android-App, mit der man Adressen suchen, den eigenen Standort anzeigen und zuvor gesuchte Orte in einem Verlauf wiederfinden kann. Sie ist als Web-App geschrieben (Vue 3 + Ionic) und wird über **Capacitor** in eine echte Android-APK verpackt – die App läuft also technisch in einer eingebetteten Android-WebView, fühlt sich aber wie eine native App an.

Warum dieser Stack?

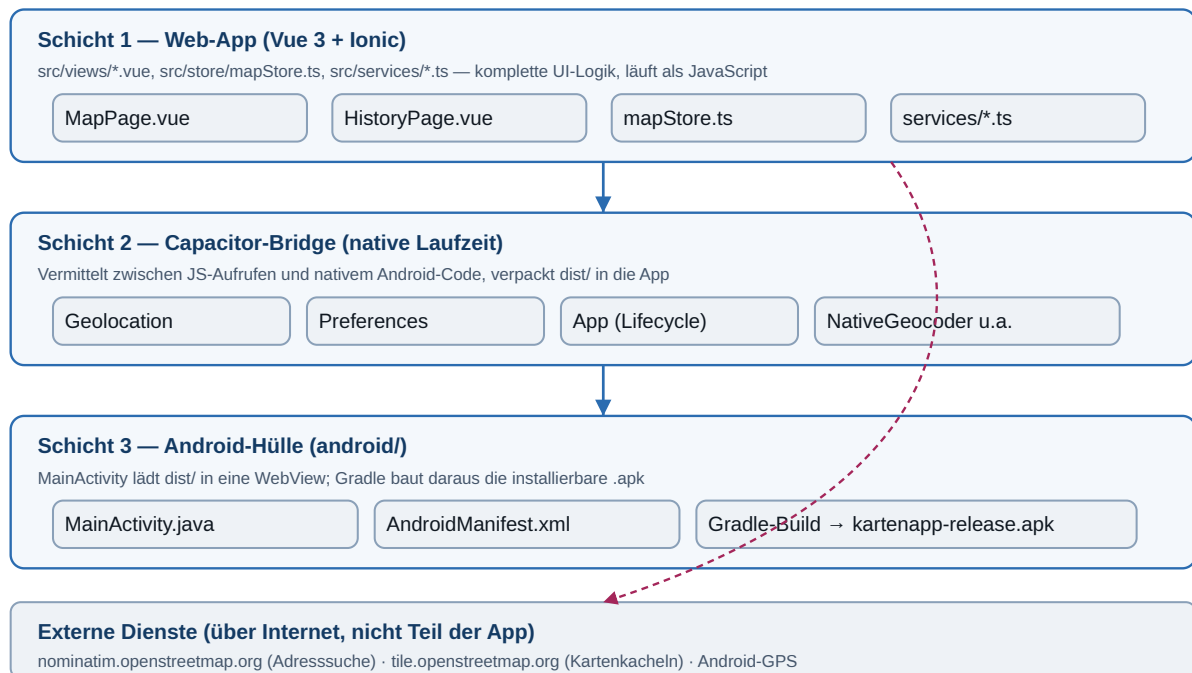
Statt eine App zweimal (Android + iOS, jeweils nativ) zu schreiben, wird hier **ein** Code-Stand (HTML/CSS/TypeScript) genutzt und plattformspezifisch verpackt. Capacitor stellt dafür Brücken („Plugins“) zu nativen Fähigkeiten bereit (GPS, Speicher, App-Lifecycle), die von JavaScript aus wie normale Funktionsaufrufe genutzt werden.

Baustein	Rolle im Projekt
Vue 3 (Composition API)	UI-Framework. Baut die Oberfläche aus reaktiven Komponenten (<code>.vue</code> -Dateien) auf.
Ionic Framework	Fertige, mobile-optimierte UI-Bausteine (Buttons, Listen, Tabs, Toasts ...) im nativen App-Look.
TypeScript	Typisiertes JavaScript – fängt viele Fehler schon beim Programmieren ab (<code>vue-tsc</code> prüft das vor jedem Build).
Vite	Build-Werkzeug/Dev-Server. Bündelt den Code für Produktion nach <code>dist/</code> und bietet Hot-Reload in der Entwicklung.
Vue Router / Ionic Vue Router	Regelt die Navigation zwischen den zwei Haupt-Ansichten (Karte, Verlauf).
Leaflet + <code>@vue-leaflet/vue-leaflet</code>	Interaktive Kartenbibliothek. Zeigt OpenStreetMap-Kacheln, Marker, Zoom/Pan.
Nominatim (OpenStreetMap)	Kostenloser Geocoding-Webdienst: wandelt Adresstext in Koordinaten um (und umgekehrt für Vorschläge).
Capacitor 8	Native Laufzeitumgebung/Bridge. Verpackt die Web-App als Android-App und stellt Plugin-APIs bereit.
Capacitor-Plugins	<code>Geolocation</code> (GPS), <code>Preferences</code> (persistenter Key-Value-Speicher), <code>App</code> (Lifecycle-Events), <code>Haptics</code> , <code>Keyboard</code> , <code>Status-Bar</code> , <code>@capgo/nativegeocoder</code> (natives Geocoding).
Android / Gradle	Natives Hüllprojekt in <code>android/</code> , das die Web-Assets einbettet und die installierbare <code>.apk</code> erzeugt.
ESLint / Vitest / Cypress	Codequalität (Linting), Unit-Tests bzw. End-to-End-Tests (aktuell nur Grundgerüst vorhanden).

Die App-ID im Android-Projekt ist weiterhin `io.ionic.starter` (Standardwert des Ionic-Starters, nie umbenannt). Der sichtbare App-Name ist „Karten App“, die erzeugte Datei heißt `kartenapp-release.apk`.

2 Architektur – wie die Schichten zusammenhängen

Man kann sich das Projekt in drei Schichten vorstellen, die aufeinander aufbauen:



In der Praxis bedeutet das: Wenn Sie im Code etwas ändern (z. B. in `MapPage.vue`), betrifft das **nur Schicht 1**. Damit die Änderung auf dem Handy sichtbar wird, muss der Web-Build neu erzeugt (`npm run build`) und in Schicht 3 kopiert werden (`npx cap sync android`), bevor die APK neu gebaut wird. Änderungen an Schicht 3 (native Android-Dateien) sind nur nötig, wenn z. B. neue Berechtigungen oder native Plugins hinzukommen.

Zusammenspiel beim Start der App

- 1 **MainActivity** (Android) startet eine `BridgeActivity` von Capacitor und lädt die in `android/app/src/main/assets/public` enthaltenen Web-Dateien (Kopie von `dist/`) in eine WebView.
- 2 Im WebView startet `main.ts`: Es erzeugt die Vue-App, bindet Ionic und den Router ein.
- 3 Bevor die App sichtbar wird, lädt `loadMapStore()` gespeicherte Kartenposition, Suchverlauf usw. aus dem persistenten Speicher (`Capacitor Preferences`, nativ als `SharedPreferences` unter Android).
- 4 Der Router leitet auf `/tabs/map` um → `TabPage.vue` rendert die Tab-Leiste und lädt `MapPage.vue`.
- 5 `MapPage.vue` baut die Leaflet-Karte auf und zeigt ggf. bereits gespeicherte Marker/Position an.

3 Projektstruktur (Ordner & Dateien im Überblick)

```
Fallstudie_Genz/
├── README.md
├── karten-a-pp/
│   ├── src/
│   │   ├── main.ts
│   │   ├── App.vue
│   │   ├── router/
│   │   │   └── index.ts
│   │   ├── store/
│   │   │   └── mapStore.ts
│   │   ├── views/
│   │   │   ├── TabsPage.vue
│   │   │   ├── MapPage.vue
│   │   │   └── HistoryPage.vue
│   │   ├── services/
│   │   │   ├── geocoding.ts
│   │   │   └── location.ts
│   │   └── theme/
│   │       └── variables.css
│   ├── android/
│   │   ├── app/
│   │   │   ├── build.gradle
│   │   │   ├── kartenapp-release.keystore
│   │   │   └── src/main/
│   │   │       ├── AndroidManifest.xml
│   │   │       ├── java/io/ionic/starter/
│   │   │       │   └── MainActivity.java
│   │   │       └── assets/public/
│   │   ├── build.gradle
│   │   ├── variables.gradle
│   │   └── keystore.properties
│   ├── dist/
│   ├── tests/e2e/
│   ├── capacitor.config.ts
│   ├── vite.config.ts
│   ├── package.json
│   └── README.md
```

Git-Repo-Wurzel
Kurzanleitung (Duplikat von karten-a-pp/README.md)
Das eigentliche App-Projekt
Web-App-Quellcode (Vue/TypeScript) – Schicht 1
Einstiegspunkt: Vue-App erzeugen & mounten
Wurzelkomponente, App-Lifecycle (Pause-Handling)
Seitenrouten (/tabs/map, /tabs/history)
Globaler reaktiver Zustand + Persistierung
Tab-Leiste (Karte / Verlauf)
Hauptseite: Karte, Suche, Standort
Liste vergangener Suchanfragen
Adresse $\rightleftharpoons$ Koordinaten (Nominatim + nativ)
GPS-Standortabfrage
Ionic-Farbvariablen (aktuell Standardwerte)
Natives Android-Hüllprojekt – Schicht 3
App-Modul-Build inkl. Signing-Konfiguration
Signier-Schlüssel (NICHT in Git)
Berechtigungen, Activity-Deklaration
Lädt die WebView (BridgeActivity)
Ziel von "cap sync" – Kopie von dist/
Projekt-weite Gradle-Konfiguration
SDK-Versionen, Bibliotheksversionen
Keystore-Zugangsdaten (NICHT in Git)
Produktions-Build von Vite (generiert, nicht editieren)
Cypress End-to-End-Test-Grundgerüst
App-ID, App-Name, Web-Verzeichnis für Capacitor
Build-/Dev-Server-Konfiguration, "@"-Alias
Abhängigkeiten & npm-Skripte
Ausführliche Bau-/Release-Anleitung

Faustregel: Alles, was das Aussehen/Verhalten der App betrifft, steht in `src/`. Alles unter `android/` betrifft nur die native Verpackung (Berechtigungen, App-Icon, Signierung, SDK-Versionen) und muss nur selten angefasst werden. `dist/` wird bei jedem `npm run build` automatisch neu erzeugt.

4 State Management & Datenfluss (mapStore)

Statt einer schweren State-Management-Bibliothek (z. B. Pinia/Vuex) nutzt das Projekt ein einziges, handgeschriebenes **reaktives Objekt** in `src/store/mapStore.ts` (via Vues `reactive()`). Jede Komponente, die dieses Objekt importiert, sieht automatisch Änderungen – ganz ohne Zusatz-Bibliothek.

Der Store hält:

Feld	Bedeutung
<code>center, zoom</code>	Aktuelle Kartenposition/Zoomstufe – wird live mit der Leaflet-Karte synchronisiert.
<code>history</code>	Liste vergangener Adresssuchen (max. 50 Einträge, neueste zuerst).
<code>pendingFocus</code>	„Merkzettel“: Ort, zu dem die Karte springen soll, sobald sie sichtbar ist (z. B. nach Klick auf Verlaufseintrag von der History- zur Map-Seite).
<code>searchMarker / userMarker</code>	Koordinaten für den roten Such-Marker bzw. blauen Standort-Marker.
<code>geoHintShown</code>	Merkt sich, ob der Hinweis „Erste Standortsuche dauert länger“ schon gezeigt wurde.
<code>loaded</code>	Schutz-Flag: verhindert, dass vor dem ersten Laden aus dem Speicher versehentlich Default-Werte zurückgeschrieben werden.

Persistierung

Kartenposition, Standort-Marker und Verlauf werden automatisch im geräteseitigen `Capacitor Preferences`-Speicher gesichert (entspricht unter Android `SharedPreferences`), damit sie einen App-Neustart überleben:

- Ein `watch()` auf Center/Zoom/Standort-Marker speichert Änderungen **debounced** (300 ms Verzögerung), damit nicht bei jedem Kartenpixel-Ruckeln geschrieben wird.
- `flushMapState()` wird beim `pause`-Event der App (App.vue) sofort ausgeführt – falls Android den Prozess im Hintergrund beendet, bevor der debounce-Timer abgelaufen ist, geht so nichts verloren.
- Der Verlauf wird bei jeder Änderung sofort (ohne debounce) gespeichert, da Hinzufügen/Löschen seltener vorkommt.

Der „pendingFocus“-Mechanismus

Klickt man in der **Verlaufsliste** auf einen Eintrag, ist die Kartenseite meist gar nicht aktiv/gemountet. Anstatt die Karte direkt zu steuern (die es in dem Moment nicht gibt), merkt sich der Store nur die Zielkoordinaten (`requestFocus`). Sobald `MapPage.vue` anschließend sichtbar wird, liest sie diesen Wert per `consumePendingFocus()` aus und springt dorthin. Das entkoppelt die beiden Seiten sauber voneinander.

5 Datei-Dokumentation – Frontend (src/)

src/main.ts

EINSTIEGSPUNKT

Erstellt die Vue-App-Instanz, bindet Ionic (`IonicVue`) und den Router ein, importiert alle Ionic-Basis-Stylesheets. Wartet mit `Promise.all([router.isReady(), loadMapStore()])` darauf, dass sowohl der Router als auch der gespeicherte Kartenzustand bereitstehen, **bevor** die App gemountet wird – so gibt es keinen sichtbaren „Sprung“ von Default- zu gespeicherter Position.

src/App.vue

WURZELKOMPONENTE

Enthält nur das Grundgerüst (`ion-app` + `ion-router-outlet`). Registriert beim Mounten einen Listener auf das native `pause` -Event (App geht in den Hintergrund) und ruft dann `flushMapState()` auf, damit ausstehende, debounced Speicherungen nicht verloren gehen, falls Android den Prozess danach beendet.

src/router/index.ts

ROUTING

Definiert zwei Tab-Routen unterhalb von `/tabs/`: `map` und `history`, beide lazy-geladen (`() => import(...)`), damit ihr Code erst beim ersten Aufruf nachgeladen wird. Root-Pfad `/` leitet automatisch auf `/tabs/map` um.

src/store/mapStore.ts – globaler Zustand & Persistierung

Details zum Inhalt siehe Kapitel 4. Wichtigste Funktionen:

Funktion	Zweck
<code>loadMapStore()</code>	Lädt beim App-Start Kartenzustand, Verlauf und Geo-Hinweis-Flag aus dem persistenten Speicher; setzt <code>loaded = true</code> am Ende (schaltet das Speichern erst danach scharf).
<code>persistMapState()</code>	Schreibt Center/Zoom/Standort-Marker als JSON in <code>Preferences</code> (privat, nur intern über den <code>watch()</code> -Debounce aufgerufen).
<code>flushMapState()</code>	Erzwingt sofortiges Speichern, umgeht den Debounce (Aufruf bei App-Pause).
<code>addHistoryEntry(entry)</code>	Fügt eine neue Suche vorne in die Verlaufsliste ein, kappt bei 50 Einträgen, speichert sofort.
<code>removeHistoryEntry(id)</code>	Löscht einen Verlaufseintrag; entfernt zusätzlich den Such-Marker von der Karte, falls er zu genau diesem Eintrag gehörte.
<code>markGeoHintShown()</code>	Setzt das Flag, damit der Erstnutzer-Hinweis zur Standortsuche nur einmal erscheint.
<code>requestFocus(focus)</code>	Hinterlegt Zielkoordinaten, zu denen die Karte beim nächsten Anzeigen springen soll.
<code>consumePendingFocus()</code>	Liest die hinterlegten Zielkoordinaten aus <i>und löscht sie</i> (einmalige Anwendung).

src/views/TabsPage.vue – Tab-Navigation

Sehr schlanke Komponente: rendert nur die untere Tab-Leiste mit zwei Reitern („Karte“, „Verlauf“) und einen `ion-router-outlet`, in dem die jeweils aktive Unterseite angezeigt wird. Enthält keine eigene Logik.

src/views/MapPage.vue – Hauptseite (Karte, Suche, Standort)

Die zentrale und umfangreichste Seite der App. Zeigt eine Leaflet-Karte (über `<l-map>` aus `@vue-leaflet/vue-leaflet`) mit OpenStreetMap-Kacheln, eine Adress-Suchleiste mit Live-Vorschlägen und einen Button zur Standortbestimmung.

Funktion	Zweck
<code>onMapReady(map)</code>	Wird von Leaflet aufgerufen, sobald die Karte fertig initialisiert ist. Speichert die rohe Leaflet-Instanz, ruft <code>invalidateSize()</code> auf (behebt Größenberechnungsfehler bei anfänglich unsichtbarem Container) und wendet einen eventuell wartenden „pendingFocus“ an.
<code>onMapCenterUpdate(center)</code>	Reagiert auf manuelles Verschieben der Karte durch den Nutzer und schreibt die neue Mitte in den Store (für Persistierung).
<code>focusMap(lat, lng)</code>	Springt die Karte direkt (ohne Animation) zu einer Zielposition und zoomt mindestens auf Stufe 16. Nutzt <code>setView(..., {''animate: false {''}})</code> , um Ruckler/„Hänger“ durch konkurrierende Animationen zu vermeiden (siehe Hinweis unten).
<code>applyPendingFocus()</code>	Holt einen ggf. hinterlegten Zielort aus dem Store (siehe Kapitel 4) und springt per <code>focusMap()</code> dorthin, setzt den Such-Marker.
<code>onSearchInput()</code>	Reagiert auf Tastatureingabe im Suchfeld; startet nach 350 ms Pause (Debounce) und ab 7 Zeichen die Vorschlagssuche.
<code>fetchSuggestions(query)</code>	Ruft <code>searchAddressSuggestions()</code> auf (Nominatim), bricht eine noch laufende vorherige Anfrage über <code>AbortController</code> ab, um veraltete Ergebnisse zu vermeiden.
<code>selectSuggestion(suggestion)</code>	Übernimmt einen angeklickten Vorschlag: Karte springt hin, Marker wird gesetzt, Eintrag landet im Verlauf.
<code>onSearch()</code>	Wird bei Enter/Suchen-Klick ausgelöst: geokodiert den eingegebenen Text über <code>geocodeAddress()</code> , zeigt bei Fehlschlag einen Toast.
<code>onLocate()</code>	Fragt per <code>getCurrentPosition()</code> den GPS-Standort ab, zeigt beim allerersten Mal einen erklärenden Hinweis-Toast, springt zur Position und setzt den blauen Standort-Marker.
<code>onClearSearch()</code> / <code>onSearchBlur()</code> / <code>clearSuggestions()</code>	Räumen Suchvorschläge/Marker auf, wenn das Suchfeld geleert oder verlassen wird (mit kurzer Verzögerung, damit ein Klick auf einen Vorschlag noch ankommt).
<code>presentToast(message)</code>	Kleiner Hilfsbaustein zum Anzeigen einer Ionic-Toast-Meldung (2,5 s).

Performance-Hinweis (behoben am 11.08.2026): Ursprünglich lief beim Springen zu einem Ort (Standort-Button, Suchtreffer, Verlaufseintrag) zusätzlich zur direkten `setView()` -Bewegung eine zweite, von `vue-leaflet` ausgelöste animierte Kamerabewegung – ausgelöst dadurch, dass `mapStore.center` / `mapStore.zoom` direkt danach ebenfalls verändert wurden und das als reaktive Props der Kartenkomponente erneut eine (animierte) Bewegung anstieß. Zwei konkurrierende Animationen kurz hintereinander führten besonders nach vorherigem manuellem Verschieben/Zoomen zu spürbarem Hängen. Behoben durch `:no-blocking-animations="true"` auf `<L-map>` sowie `{{''} animate: false {'}'}` in `focusMap()` — die Karte springt jetzt sofort, ohne Animation.

src/views/HistoryPage.vue – Suchverlauf

Zeigt die im Store hinterlegte Verlaufsliste als „swipe-to-delete“-Liste (`ion-item-sliding`). Klick auf einen Eintrag ruft `requestFocus()` auf und navigiert zurück zur Kartenseite (`router.push('/tabs/map')`) — die Karte springt dort dank `pendingFocus` -Mechanismus automatisch zum gewählten Ort. Ein Wisch-Button pro Zeile löscht den Eintrag über `removeHistoryEntry()` .

src/services/geocoding.ts – Adresse ≠ Koordinaten

Funktion	Zweck
<code>geocodeAddress(address)</code>	Haupteinstieg für die „Suchen“-Aktion: Versucht auf echten Android-Geräten zuerst natives Geocoding (schneller, funktioniert teils offline); schlägt das fehl oder läuft die App im Browser, wird automatisch auf die Nominatim-Weboberfläche zurückgefallen.
<code>geocodeNative(query)</code>	Nutzt das Plugin <code>@capgo/nativegeocoder</code> , das die Geocoding-Fähigkeit des Betriebssystems anspricht.
<code>formatNativeAddress(a)</code>	Baut aus den strukturierten nativen Adressfeldern (Straße, Hausnr., PLZ, Ort) einen lesbaren Anzeigetext.
<code>searchAddressSuggestions(query, options)</code>	Liefert Live-Vorschläge während der Eingabe über die Nominatim- <code>/search</code> -API (Standardlimit 3 Treffer). Unterstützt Abbruch laufender Anfragen via <code>AbortSignal</code> .
<code>geocodeWithNominatim(query)</code>	Web-Fallback: fragt Nominatim nach genau einem besten Treffer.

Nominatim verlangt laut Nutzungsbedingungen eine erkennbare Anfragequelle. Da `fetch()` das `User-Agent` -Header nicht erlaubt (vom Browser blockiert), wird stattdessen `referrerPolicy: 'origin'` gesetzt.

src/services/location.ts – GPS-Standortabfrage

Funktion	Zweck
<code>getCurrentPosition()</code>	Fragt zunächst die Standort-Berechtigung an (<code>Geolocation.requestPermissions()</code>), wartet danach 300 ms (umgeht einen bekannten Android-Plugin-Bug, bei dem eine frisch erteilte Berechtigung intern noch nicht propagiert ist), und ruft dann <code>Geolocation.getCurrentPosition()</code> mit hoher Genauigkeit, 45 s Timeout und bis zu 10 s altem Cache-Fix auf.

6 Datei-Dokumentation – Android-Projekt

Der Ordner `android/` ist ein vollständiges, eigenständiges Gradle/Android-Projekt, wie es Capacitor beim Erstellen der Plattform generiert. Er wird nicht von Hand geschrieben, sondern über `npx cap sync android` mit dem Web-Build und den installierten Capacitor-Plugins synchron gehalten. Nur wenige Dateien darin sind für dieses Projekt manuell angepasst:

`android/app/src/main/AndroidManifest.xml`

APP-MANIFEST

Deklariert die `MainActivity` als Startpunkt (Launcher-Icon), einen `FileProvider` (für Datei-Freigaben, von Capacitor-Plugins benötigt) sowie die angeforderten Berechtigungen: `INTERNET` (Kartenkacheln/Nominatim), `ACCESS_COARSE_LOCATION` und `ACCESS_FINE_LOCATION` (GPS) plus das Feature `android.hardware.location.gps`.

`android/app/src/main/java/io/ionic/starter/MainActivity.java`

EINSTIEGSPUNKT (NATIV)

Eine einzeilige Klasse, die von Capacitors `BridgeActivity` erbt. Diese Basisklasse kümmert sich um das Erzeugen der WebView, das Laden von `assets/public/index.html` und die gesamte Plugin-Verdrahtung – eigener Code ist hier normalerweise nicht nötig.

`android/app/build.gradle`

APP-MODUL-BUILD

Definiert `applicationId` (`io.ionic.starter`), min/target-SDK-Version (aus `variables.gradle`), sowie die **Signing-Konfiguration** für Release-Builds: Liest Keystore-Pfad und Passwörter aus `android/keystore.properties` (falls vorhanden) und signiert damit automatisch jede `assembleRelease` -APK. Benennt die Ausgabedatei über `outputFileName` in `kartenapp-debug.apk` / `kartenapp-release.apk` um.

`android/build.gradle` & `android/variables.gradle`

PROJEKT-WEITE KONFIGURATION

Legen Repositories (Google, Maven Central), die Android-Gradle-Plugin-Version sowie zentral gepflegte SDK-/Bibliotheksversionen fest (`compileSdk` / `targetSdk` = 36, `minSdk` = 24, also ab Android 7.0).

capacitor.config.ts (im Projekt-Root, nicht in android/)

CAPACITOR-KONFIGURATION

Verknüpft Web- und Android-Welt: **appId** (muss mit der Android **applicationId** übereinstimmen), **appName** („Karten App“) und **webDir** (**dist** – welcher Ordner beim **cap sync** nach **android/app/src/main/assets/public** kopiert wird).

Sicherheitshinweis: **android/app/kartenapp-release.keystore** und **android/keystore.properties** sind bewusst per **.gitignore** vom Git-Repository ausgeschlossen. Geht der Keystore verloren, lassen sich bereits installierte Apps auf Testgeräten nicht mehr per Update-APK aktualisieren (Android verlangt dieselbe Signatur) – Neuinstallation wäre nötig. Beide Dateien daher separat sichern (Passwort-Manager/verschlüsseltes Backup).

7 Build- & Release-Workflow

Entwicklung (schnelles Iterieren im Browser)

```
npm run dev          # Vite-Dev-Server mit Hot-Reload, http://localhost:5173
```

Läuft direkt im Desktop-Browser, ohne Android – ideal für UI-Arbeit. GPS/native Plugins verhalten sich hier über Browser-APIs (eingeschränkter als am echten Gerät).

Qualitätsprüfungen

```
npm run lint          # ESLint – Stil-/Fehlerprüfung
npm run test:unit     # Vitest – Unit-Tests
npm run test:e2e      # Cypress – End-to-End-Tests im Browser
```

Neue Android-APK erzeugen (Kompletter Ablauf)

- 1 **Web-Build erzeugen:** `npm run build` (führt zuerst `vue-tsc` -Typecheck aus, dann Vite-Produktionsbuild nach `dist/`).
- 2 **In Android-Projekt kopieren:** `npm run cap sync android` (kopiert `dist/` + aktualisiert alle Capacitor-Plugins in `android/`).
- 3 **APK bauen:** im Ordner `android/` entweder
`./gradlew assembleDebug` (unsigned, nur zum Testen) oder
`./gradlew assembleRelease` (signiert, installierbar – benötigt Keystore).
- 4 **Ergebnis:** `android/app/build/outputs/apk/release/kartenapp-release.apk` (bzw. `debug/kartenapp-debug.apk`).
- 5 **Auf Gerät installieren:** `adb install -r android/app/build/outputs/apk/release/kartenapp-release.apk` (`-r` ersetzt eine vorhandene Installation, Update-fähig dank gleichbleibender Signatur).

Schritte 1 und 2 müssen nach **jeder** Code-Änderung wiederholt werden, die in eine neue APK soll – ein reines `./gradlew assembleRelease` ohne vorherigen `build / sync` würde nur den zuletzt kopierten (alten) Stand erneut verpacken.

8 Glossar wichtiger Begriffe

Capacitor

Native Laufzeitumgebung von Ionic, die eine Web-App (HTML/JS) in eine echte iOS-/Android-App verpackt und über „Plugins“ Zugriff auf native Gerätefunktionen (GPS, Speicher, Kamera ...) gibt.

WebView

Eine in eine native App eingebettete Mini-Browser-Komponente, in der die Web-App tatsächlich läuft. Auf Android ist das Chrome-basiert.

Ionic Framework

UI-Komponentenbibliothek (`ion-button` , `ion-list` usw.), die mobile-typische Bedienelemente im plattformgerechten Look liefert – unabhängig davon, ob Vue, React oder Angular als Basis genutzt wird (hier: Vue).

Vue Composition API (`<script setup>`)

Moderner Vue-3-Schreibstil: Zustand (`ref` / `reactive`) und Funktionen werden direkt in der Komponente deklariert, statt in einem `data` / `methods` -Objekt (Options API).

Reaktivität

Vue-Mechanismus, bei dem sich die Oberfläche automatisch aktualisiert, sobald sich zugrunde liegende Daten (z. B. `mapStore.center`) ändern – ohne manuelles „Neu zeichnen“.

Leaflet

Leichtgewichtige JavaScript-Bibliothek für interaktive Karten (Zoom, Pan, Marker). Zeigt hier Kartenkacheln von OpenStreetMap an.

Geocoding / Nominatim

„Geocoding“ = Adresstext in Koordinaten umwandeln (und umgekehrt „Reverse Geocoding“). **Nominatim** ist der kostenlose, öffentliche Geocoding-Dienst von OpenStreetMap, den diese App für Adresssuche/-vorschläge nutzt.

Debounce

Technik, um eine Aktion (z. B. Netzwerk-Anfrage, Speichern) erst auszuführen, nachdem eine kurze Zeit lang keine weitere Änderung mehr kam – verhindert unnötig viele Anfragen/Schreibvorgänge bei schneller Eingabe.

AbortController

Browser-API, mit der eine laufende `fetch()` -Anfrage aktiv abgebrochen werden kann – hier genutzt, um veraltete Vorschlags-Anfragen zu verwerfen, wenn der Nutzer weitertippt.

Capacitor Preferences

Plugin für einfachen, persistenten Key-Value-Speicher auf dem Gerät (entspricht unter Android `SharedPreferences`) – hier für Kartenzustand und Suchverlauf genutzt.

Gradle

Build-Werkzeug der Android-/Java-Welt, mit dem aus dem `android/` -Projekt die fertige `.apk` -Datei erzeugt wird (Analogon zu `npm run build` auf der Web-Seite).

Keystore / Signing

Jede installierbare Android-App muss digital signiert sein. Der „Keystore“ enthält den privaten Schlüssel dafür. Nur APKs mit **derselben** Signatur können ein bereits installiertes App-Update ersetzen.

vue-tsc

TypeScript-Typprüfung speziell für `.vue`-Dateien; läuft automatisch vor jedem Produktions-Build (`npm run build`) und bricht bei Typfehlern ab.